Counting System Complexity, Mental Load, and Power per Complexity

Date: 2026-04-07

What This Is

This is an attempt to lay the whole thing open.

The problem starts with a familiar blackjack argument. One count is supposed to be "easy," another is "strong," another is "not worth the trouble," and a fourth is "great if you can really use it." Most of that is directionally sensible and mathematically foggy. The point of this framework is to stop leaving the comparison there.

What we wanted was a standard way to answer three separate questions:

1. How mentally demanding is a counting system?
2. What exactly is creating that burden?
3. Once you know the burden, how much practical power are you getting back for it?

That is why this ends up as a two-part exercise.

Part one is a complexity and mental-load framework. It scores the system itself: tag burden, count-conversion burden, running-count envelope, side-count burden, negative-territory burden, and so on.

Part two is a power layer. It takes a standardized set of long-run simulation outputs and asks the second question that always follows the first: given that burden, what are you actually getting back?

That second step is where the "miles per gallon" idea comes in. A system can be very strong in raw terms and still be a lousy bargain if the extra power comes at a huge increase in operating burden. Another system can be weaker in absolute terms and still be the better overall bargain because it gives you much more performance per unit of complexity.

This report walks through the framework from the original spec to the current implementation, then shows exactly how the current study numbers are produced.

Validation Posture Disclaimer

It is important at the outset that we make our Blackjack Wonk (BJW) validation disclaimer crystal clear.

BJW is currently operating under a two-track validation posture:

- Track A = BJW versus CVData parity
- Track B = BJW versus published or otherwise defensible math and rulebook checks

Current posture:

- Track B is accepted as the correctness baseline
- Track A remains a documented residual gap

In plain English, that means:

- the BJW framework math, the scorer math, and the internal comparative logic in this report are being presented as Track B-defensible. We are satisfied that the framework, scorer, and rulebook checks are defensible under Track B
- the absolute counted EV, SCORE, DI, and N0 values in BJW are **not** being claimed here as external-parity-equivalent to CVData or CVCX
- relative comparisons within the same BJW engine, under the same rules, spreads, and study basis, carry much stronger confidence than any claim that BJW's absolute counted outputs numerically match QFIT commercial tools

That is not a throwaway disclaimer. It is our current validation posture, and will be until we are satisfied that our results come close to CVData. This report needs to be read inside it.

It is also worth stating, briefly, why Track A may remain out of reach even after a serious parity campaign. Matching CVData as closely as possible was a high priority for BJW, and we worked for months trying to get the two systems to line up. We ran an iterative campaign, fixed BJW-side defects as we found them, reran the suite, and still came back with gaps that were mixed in both size and direction rather than collapsing under a single correction. The current best explanation is some combination of occupancy and true-count-frequency shape mismatch plus limited observability into CVData's detailed internal setup and runtime choices. In other words, this may not be a matter of "one more bug and the numbers will snap into place." It may be a black-box limit. That is not offered as an excuse. It is offered so the reader understands that the project did, in fact, push hard on parity before deciding to treat Track B as the working correctness baseline.

There is also a practical reason this work was done in BJW instead of trying to muscle the whole study through CVData by hand. BJW is scriptable end to end. The counting files, spreads, compatibility map, manifest generation, simulation launchers, long-run campaign receipts, result pullback, and scoring pass can all be batch-driven and reproduced. That matters when the study is not one sim but a stack of billion-round runs, all under a fixed protocol, followed by derived scoring, validation, and re-ranking. CVData remains valuable to us as an external reference. BJW was simply the better tool for this particular job because we needed a large automated research workflow with inspectable inputs, repeatable runs, and batch scoring.

First Principle: Fix the Game

The framework is not trying to estimate mental burden in every possible blackjack situation. It is a reference-condition comparison.

The reference game in the original framework is:

- 6 decks

- S17
- DOA
- DAS
- LS
- RSA
- peek on ace or ten
- 75 percent penetration
- minimum 1 billion rounds for simulation-derived components

That matters because complexity is not context-free. Heads-up is easier than a packed table. Pitch is not a shoe game. A quarter-deck true-count habit is not the same burden as a whole-deck true-count habit. The only honest way to compare systems is to freeze the game first.

In the current study, every fully scored row is aligned to that reference game, and the scorer rejects a row if the attached BJW `results.json` does not match it.

The Core Idea: Two Different Kinds of Burden

The framework splits mental load into two buckets.

Instantaneous load is what you are doing right now. Card comes out, dealer shows a card, a player hits or doubles, somebody splits, the count moves, the divisor changes, you have to act. This is per-card and per-decision burden.

Sustained load is what you are carrying for the whole shoe. How wide is the running-count envelope? How much negative-territory bookkeeping are you living in? Are you carrying a side count? Are you living off an IRC? Are you tracking a key count or a pivot?

That split matters because different systems are hard in different ways. Hi-Lo asks you to do true-count arithmetic all shoe long. KO asks you to live inside a more complicated running-count state. Those are not the same kind of burden, even if a final scalar score eventually places them in the same neighborhood.

The Eleven Raw Components

The framework uses eleven raw components, `C1` through `C11`.

ID	Name	Formula or rule	Why it matters
`C1`	Distinct non-zero tag values	Count distinct non-zero tags in the system definition	More distinct tags means more card-value bookkeeping
`C2`	Tag value range	`max_tag -`	Wider value space

ID	Name	Formula or rule	Why it matters
		`min_tag`	means larger mental arithmetic span
`C3`	Non-standard tag resolution	Count ranks with fractional or suit-aware tags	Rank alone is no longer enough; you need extra resolution
`C4`	TC conversion required	`1` if the system requires true count conversion, else `0`	True-count conversion is real extra work
`C5`	Divisor changes per shoe	`floor(decks_in_play / precision_unit)` in the general formula	More divisor changes means more moving arithmetic targets
`C6`	RC range at 99th percentile	`p99(RC) - p1(RC)` from `count_histogram.RC` at `bet_decision`	Measures the normal working-memory envelope
`C7`	Side count severity	Tiered `0`, `1`, or `2`	Side counts add both parallel memory load and reconciliation load
`C8`	Starting count magnitude	`abs(IRC)` at the study deck count	Big negative starts are not free
`C9`	Proportion of shoe in negative territory	`P(RC < 0)` from the same RC histogram	Living in negative count space has a real cognitive feel
`C10`	Key count applies	`1` if the operational variant uses a key count, else `0`	Key counts add threshold tracking
`C11`	Pivot applies	`1` if the operational variant uses a pivot, else `0`	Pivots add another threshold concept to manage

Exact raw formulas

For clarity, here are the raw formulas in notation.

Let `t(r)` be the tag value assigned to rank `r`.

Then:

```
C1 = number of distinct non-zero values in { t(r) }

C2 = max_r t(r) - min_r t(r)

C3 = number of affected ranks where the tag is fractional or suit-aware
```

```
C4 = 1 if true-count conversion is operationally required, else 0

C5 = floor(decks_in_play / precision_unit)
```

with:

```
decks_in_play = deck_count * penetration_fraction

precision_unit in {1.0, 0.5, 0.25}
```

For the current reference study, true-count systems scored on a half-deck basis use `C5 = 9`.

In the reference game:

```
decks_in_play = 6 * 0.75 = 4.5

floor(4.5 / 0.5) = 9
```

For the simulation-derived terms:

```
C6 = percentile_99(RC_histogram) - percentile_1(RC_histogram)

C9 = sum_{rc < 0} w(rc) / sum_{all rc} w(rc)
```

where `w(rc)` is the histogram weight in the `count_histogram.RC` distribution sampled at `bet_decision`.

For the declaration-driven terms:

```
C7 = 0  if no side count
   = 1  if side count is mainly betting or insurance support
   = 2  if side count materially affects playing decisions or multi-parameter play

C8 = abs(starting_count)

C10 = 1 if key count applies, else 0

C11 = 1 if pivot applies, else 0
```

Why these particular pieces were chosen

The framework is trying to capture burden that is countable, reproducible, and not just somebody's impression.

`C1`, `C2`, and `C3` are structural. They come straight out of the tag set.

`C4` and `C5` are arithmetic burden. A true-count system is doing more work than a running-count-only system. A quarter-deck player is doing more divisor maintenance than a whole-deck player.

`C6`, `C8`, `C9`, `C10`, and `C11` are state-management burden. This is the difference between "simple tags" and "simple to live with." KO is the clean example. The tags are easy. The operating state is not as easy as the tags make it look.

`C7` is both. A side count is immediate work and persistent work. That is why the framework counts it in

both sub-scores.

One Explicit Modeling Choice

The framework tries hard not to introduce arbitrary weighting. It still makes one open, explicit modeling choice:

`C7` is counted twice.

The final score is built from twelve slots, not eleven, because `C7` appears once in the instantaneous bucket and once in the sustained bucket.

That is not some discovered law of cognition. It is a declared modeling decision. The reason is straightforward: a side count does not just add one more number. It adds one more number that has to be maintained continuously and then reconciled at decision time.

Doubling `C7` is an explicit modeling judgment, not an empirical fact. The important thing is to show that judgment openly, explain the reason for it, and let the reader decide whether it is reasonable.

From Raw Components to a Single Complexity Score

The raw values live on different scales, so they have to be normalized before they can be combined.

For each component:

```
C_in(system) = C_i_raw(system) / max_j C_i_raw(system_j)
```

That means each normalized component sits between `0` and `1`, relative to the current study set.

The two sub-scores are:

```
Instantaneous = C1n + C2n + C3n + C4n + C5n + C7n

Sustained = C6n + C7n + C8n + C9n + C10n + C11n
```

The final normalized complexity score is:

```
final_score =
  (C1n + C2n + C3n + C4n + C5n + C7n + C6n + C7n + C8n + C9n + C10n + C11n) / 12
```

The stable raw companion is:

```
raw_complexity_total =
  C1 + C2 + C3 + C4 + C5 + C6 + C7 + C8 + C9 + C10 + C11
```

This is an unweighted profile sum, so `C7` appears once in `raw_complexity_total`. The extra weight for `C7` is applied only when the framework moves from raw profile description to the final normalized burden score.

Notice the difference:

- `raw_complexity_total` is stable as long as the underlying row does not change

- `normalized_complexity_score` is study-relative and will move when the denominator set moves

That is not a defect so much as the cost of using a relative ranking.

Flags: When the Score Is Not the Whole Story

The normalized complexity score collapses eleven components into a single number. That is useful for ranking and for the miles-per-gallon ratios. It also loses information.

Two systems can land at nearly the same final score for entirely different reasons. One system might score 0.35 because of true-count arithmetic burden. Another might score 0.35 because of negative-territory life and IRC burden. Those are not the same cognitive experience even if the scalar is similar.

That is what flags are for.

A flag is set for a component whenever its raw value is non-zero. No threshold judgment is involved. If C8 is non-zero, the C8 flag is set. If C7 is non-zero, the C7 flag is set. The rule is that simple.

Reading the flags alongside the score tells you not just how much burden a system carries but where that burden lives. Two systems with similar scores but different flag profiles are harder in different ways. That distinction matters when a player is deciding which system fits their actual cognitive strengths and limits, and it is a distinction the final scalar alone cannot make.

What a "Unit of Complexity" Means Here

This needs to be stated carefully because there is no physical or well understood unit here in the way there is a dollar, a hand, or a percent.

The complexity score is dimensionless. It is a normalized burden index.

In practice there are two useful ways to think about a "unit of complexity":

1. A raw component unit, inside the native component itself

Examples:

- one additional distinct non-zero tag value in `C1`
- one additional divisor change in `C5`
- one additional IRC point in `C8`

Those raw units are real, but they are not commensurate with each other.

1. A normalized complexity point in the final score

This is the one used in the "per complexity" ratios.

Here, one full unit means `1.0` on the normalized complexity scale: the notional burden of scoring at the observed maximum on every normalized slot in the current study. Since the final score is the mean of twelve normalized slots, one fully normalized slot contributes:

```
1 / 12 = 0.083333...
```

to the final score.

So when we say:

```
SCORE per complexity = SCORE / normalized_complexity_score
```

we mean:

"How many SCORE points do I get per one normalized complexity point?"

That is not a universal constant. It is an internal efficiency ratio inside this study. It is useful, but it is not absolute.

The Current Denominators

The current study has 19 top-level systems with sourced spreads and 1 billion-round reference-game runs. In the present scored sheet, the normalization denominators are:

Component	Current denominator	Current max holder(s)
`C1`	6.0	`revere_rapc`, `revere_14_count`
`C2`	8.0	`revere_rapc`
`C3`	4.0	`halves`
`C4`	1.0	all true-count systems in the study
`C5`	9.0	all half-deck true-count systems in the study
`C6`	105.0	`revere_rapc`
`C7`	1.0	`canfield_expert`, `hi_opt_i`, `hi_opt_ii`, `omega_ii`, `revere_adv_plus_minus`, `uston_apc`
`C8`	20.0	`ko`, `reko`
`C9`	0.876984018	`ko`, `reko`
`C10`	1.0	`ko`, `reko`
`C11`	1.0	`ko`, `red_seven`, `reko`

These are not eternal constants. They are the current maxima in the current study set.

Add a harder level-4 system or a system with a heavier side-count burden and some of those denominators will move. When they move, all normalized scores move. That is expected and must be

disclosed whenever the study set changes.

What the Scorer Appends Beyond Complexity

The current scored sheet adds a second family of fields that are not part of the complexity score itself.

These are the power-side numbers:

- `bjw_ev_per_hand_units`
- `bjw_ev_per_initial_bet_pct`
- `bjw_sd_per_hand_units`
- `bjw_avg_bet_units`
- `bjw_sim_score`
- `bjw_di`
- `bjw_n0`

And these are the "miles per gallon" columns:

- `bjw_score_per_complexity`
- `bjw_di_per_complexity`
- `bjw_ev_pct_per_complexity`
- `bjw_inverse_n0_per_complexity`

The logic is simple enough. First score the burden. Then divide the return by the burden.

Before reading any power table, keep one study restriction in view: these are base-count/base-spread runs with no authored deviation packages enabled. The study is deliberately isolating counting-and-betting burden and return, not full indexed-play ceilings. To keep the miles-per-gallon analogy, we are comparing base-model systems here. Options matter, but options are extra, and the available packages can vary widely by system.

Exact Power-Side Formulas in the Current BJW Contract

This is where basis discipline matters.

The scorer itself does not recompute BJW `SCORE`, `DI`, or `N0`. In the current output contract, `bjw_sim_score` is taken from `summary.score` when that field is present and otherwise falls back to `summary.risk_primary.score`. `bjw_di` and `bjw_n0` are copied from `summary.risk_primary`.

For counted runs, the simulator currently sets `risk_primary` on an observed per-round basis, not a per-hand basis.

The relevant runtime formulas are:

```
ev_round = sumNetUnits / rounds

variance_round = observed.variance_per_round

sd_round = sqrt(variance_round)

SCORE = 1,000,000 * ev_round^2 / variance_round

DI = 1000 * ev_round / sd_round

N0 = (sd_round / ev_round)^2
```

Those formulas are taken from the current simulator runtime contract, not from a later reporting layer.

The other appended power fields are:

```
evPerHandUnits = sumNetUnits / handsTotal

evPerInitialBetUnits = sumNetUnits / initialBetUnitsTotal

evPerInitialBetPct = 100 * evPerInitialBetUnits
```

and `avgBetUnits` is copied from the simulator's betting summary.

Important basis caveat

This means the scored sheet currently mixes two kinds of power fields:

- per-hand fields such as `bjw_ev_per_hand_units` and `bjw_sd_per_hand_units`
- per-round risk fields such as `bjw_sim_score`, `bjw_di`, and `bjw_n0`

That is not wrong as long as it is disclosed. It does mean you should not casually assume every power column in the sheet lives on the same denominator basis.

If someone wants to replicate the study in CVData or another simulator, they need to either:

- use the simulator's own published `SCORE`, `DI`, and `N0` fields if the basis is clearly stated, or
- recompute those fields from exported EV and SD on a declared common basis

The important thing is not the brand name of the simulator. The important thing is basis consistency.

The "Miles per Gallon" Ratios

What we wanted here was a single way to say not just "How strong?" but "How strong for the trouble?"

That is what the ratio columns do.

This is also the spot where the word "score" can get muddy if it is not pinned down.

In this section:

- `SCORE` in all caps means Don Schlesinger's blackjack `SCORE` metric, carried in the sheet as `bjw_sim_score`

- `complexity score` means the framework's normalized burden index, carried in the sheet as `normalized_complexity_score`

The current formulas are:

```
SCORE per complexity = SCORE / complexity score
                     = bjw_sim_score / normalized_complexity_score

DI per complexity = DI / complexity score
                  = bjw_di / normalized_complexity_score

EV% per complexity = EV% / complexity score
                   = bjw_ev_per_initial_bet_pct / normalized_complexity_score

inverse N0 per complexity = (1,000,000 / N0) / complexity score
                          = (1,000,000 / bjw_n0) / normalized_complexity_score
```

Under the current BJW risk formulas:

```
1,000,000 / N0 = SCORE
```

so `inverse N0 per complexity` and `SCORE per complexity` are mathematically the same number. Both are still useful to publish because some readers think more naturally in `SCORE`, while others think more naturally in `N0`.

If you want a literal "miles per gallon" number, `SCORE per complexity` is the cleanest single candidate. It rolls power and variance together, and it rewards a system that earns its keep.

`DI per complexity` is a good companion metric.

`EV% per complexity` is also useful, but it ignores variance, so it is not as complete.

BC, PE, and IC: What We Use and Why

The scored sheet now carries two different families of efficiency fields.

The first family is reference data copied from QFIT:

- `qfit_bc`
- `qfit_pe`
- `qfit_ic`

Those are published external values. They are not calculated from BJW simulation outputs.

The second family is computed in-house from the BJW counting definitions:

- `computed_bc`
- `computed_pe`
- `computed_ic`

plus the delta columns:

- `delta_bc_vs_qfit`
- `delta_pe_vs_qfit`
- `delta_ic_vs_qfit`

Correlation formula

All three computed metrics use Pearson correlation:

```
corr(x, y) =
  sum_i (x_i - mean(x)) (y_i - mean(y))
  -------------------------------------
  sqrt( sum_i (x_i - mean(x))^2 ) * sqrt( sum_i (y_i - mean(y))^2 )
```

The count vector is always the system's tag vector over actual card ranks in this order:

```
[A, 2, 3, 4, 5, 6, 7, 8, 9, T, J, Q, K]
```

Face cards inherit the system's ten value where the counting file uses a single `T` entry.

Computed BC

`computed_bc` uses a fixed betting-EOR target vector:

```
b =
[-0.61, 0.38, 0.44, 0.55, 0.69, 0.46, 0.28, 0.00, -0.18, -0.51, -0.51, -0.51,
-0.51]
```

So:

```
computed_bc = corr(tag_vector, b)
```

Why use it? Because this is exactly the kind of quantity BC is supposed to measure: how well the tag system lines up with betting value by card rank. The vector is printed in full here so the reader does not have to chase another file or a hidden constant to reproduce the calculation.

Computed IC

`computed_ic` uses the perfect-insurance vector:

```
i =
[4, 4, 4, 4, 4, 4, 4, 4, 4, -9, -9, -9, -9]
```

So:

```
computed_ic = corr(tag_vector, i)
```

Why use it? Because insurance is fundamentally about the ratio of ten-value cards to non-tens. This basis is unusually clean, which is why the computed `IC` values reproduce QFIT very closely.

Computed PE

This is where the ground gets less solid.

QFIT publishes `PE`, but it does not publish the exact target vector or weighting method used to generate the table values. So we had two choices:

1. leave `PE` as reference-only
2. compute a clearly labeled BJW proxy and say exactly what it is

We chose option 2.

The current `computed_pe` is built from a total-EOR vector:

```
u =
[-0.094, 0.069, 0.082, 0.110, 0.141, 0.079, 0.041, -0.008, -0.040, -0.091, -0.091,
-0.091, -0.091]
```

and then removes the betting-aligned component of that vector.

First compute the projection of `u` onto `b`:

```
proj_b(u) = ((u · b) / (b · b)) * b
```

Then compute the residual:

```
p = u - proj_b(u)
```

Then:

```
computed_pe = corr(tag_vector, p)
```

Why use this? Because it isolates the part of total card-removal signal that is not already aligned with plain betting strength. That makes it a defensible proxy for "playing information" rather than "betting information."

The PE caveat

The problem is that "defensible proxy" is not the same thing as "published PE reproduction."

In the current study:

- `computed_bc` tracks QFIT very closely
- `computed_ic` tracks QFIT very closely
- `computed_pe` does not track QFIT closely at all

That is why the repo now treats `computed_pe` as a BJW proxy, not as a claim to have replicated Griffin's or QFIT's published PE numbers.

That distinction matters. It is the difference between honest methodology and just dressing up a guess in notation.

The Current Reproduction Check

Against the current 19-system study:

- `BC` mean absolute delta vs QFIT = `0.003`
- `BC` max absolute delta vs QFIT = `0.012`
- `IC` mean absolute delta vs QFIT = `0.002`
- `IC` max absolute delta vs QFIT = `0.020`
- `PE` mean absolute delta vs QFIT = `0.533`
- `PE` max absolute delta vs QFIT = `0.673`

In plain English:

- the `BC` calculation is doing what it should be doing
- the `IC` calculation is doing what it should be doing
- the `PE` proxy is useful as an internal descriptor, but not as a QFIT replacement

Worked Example 1: Hi-Lo Complexity

From the current scored sheet, Hi-Lo has raw values:

```
C1 = 2
C2 = 2
C3 = 0
C4 = 1
C5 = 9
C6 = 32
C7 = 0
C8 = 0
C9 = 0.468485907
C10 = 0
C11 = 0
```

Using the current denominators:

```
C1n = 2 / 6   = 0.333333...
C2n = 2 / 8   = 0.25
C3n = 0 / 4   = 0
C4n = 1 / 1   = 1
C5n = 9 / 9   = 1
C6n = 32 / 105 = 0.3047619...
C7n = 0 / 1   = 0
C8n = 0 / 20  = 0
C9n = 0.468485907 / 0.876984018 = 0.534201...
C10n = 0 / 1  = 0
C11n = 0 / 1  = 0
```

So the final score is:

```
(0.333333 + 0.25 + 0 + 1 + 1 + 0
 + 0.304762 + 0 + 0 + 0.534201 + 0 + 0) / 12
= 0.285191...
```

The first six terms are the instantaneous block:

```
C1n + C2n + C3n + C4n + C5n + C7n
```

The second six terms are the sustained block:

```
C6n + C7n + C8n + C9n + C10n + C11n
```

That is the current `normalized_complexity_score` for Hi-Lo.

Worked Example 2: KO as a Different Cognitive Character

KO's tags look easy. Its operating state does not.

In the current sheet:

```
C1 = 2
C2 = 2
C4 = 0
C5 = 0
C6 = 38
C8 = 20
C9 = 0.876984018
C10 = 1
C11 = 1
```

That produces:

```
normalized_complexity_score = 0.412103...
```

So KO is not being penalized for tag difficulty or TC arithmetic. It is being penalized for negative-territory life, IRC burden, key-count burden, and pivot burden. That is exactly the kind of distinction this framework is supposed to surface.

Worked Example 3: "Miles per Gallon"

Take Hi-Lo again.

From the current scored sheet:

```
normalized_complexity_score = 0.2851913691
bjw_sim_score = 22.7251465828
bjw_di = 4.7670899491
bjw_ev_per_initial_bet_pct = 0.8705902357
```

So:

```
SCORE per complexity = 22.7251465828 / 0.2851913691 = 79.683851...

DI per complexity = 4.7670899491 / 0.2851913691 = 16.715407...

EV% per complexity = 0.8705902357 / 0.2851913691 = 3.052653...
```

Now compare that to KO:

```
KO SCORE per complexity = 20.7843754322 / 0.4121031746 = 50.434883...
```

Hi-Lo is not just stronger in raw `SCORE` than KO in this study. It is much stronger per unit of normalized complexity burden.

Revere Point Count is the most interesting current example:

```
RPC SCORE per complexity = 28.6573781370 / 0.3466718983 = 82.664266...
```

That is why it presently comes out as the best "miles per gallon" system in the current 19-system sheet.

Current 19-System "Miles per Gallon" Table

Sorted by `SCORE per complexity`:

System	Complexity	EV %	SCORE	DI	N0	SCORE / complexity	DI / complexity	EV % / complexity
revere_point_count	0.347	0.917	28.657	5.353	34,895	82.664	15.442	2.645
hilo	0.285	0.871	22.725	4.767	44,004	79.684	16.715	3.053
canfield_master	0.356	0.763	21.187	4.603	47,198	59.570	12.942	2.145
uston_adv_plus_minus	0.285	0.623	15.551	3.943	64,306	54.569	13.838	2.187
omega_ii	0.522	0.970	27.282	5.223	36,654	52.231	10.000	1.858
ko	0.412	0.813	20.784	4.559	48,113	50.435	11.063	1.973
silver_fox	0.289	0.559	13.841	3.720	72,251	47.957	12.891	1.938
revere_rapc	0.463	0.746	22.087	4.700	45,276	47.684	10.146	1.611
revere_14_count	0.442	0.713	19.900	4.461	50,252	45.064	10.102	1.614
red_seven	0.310	0.605	12.940	3.597	77,280	41.759	11.609	1.952
reko	0.412	0.719	17.008	4.124	58,795	41.272	10.007	1.744
mentor	0.359	0.536	12.917	3.594	77,418	35.978	10.011	1.494
halves	0.416	0.551	14.013	3.743	71,360	33.716	9.007	1.326

System	Complexity	EV %	SCORE	DI	N0	SCORE / complexity	DI / complexity	EV % / complexity
`uston_apc`	0.577	0.684	18.182	4.264	55,000	31.495	7.386	1.185
`revere_adv_plus_minus`	0.452	0.565	12.854	3.585	77,799	28.443	7.933	1.250
`zen_count`	0.356	0.433	9.481	3.079	105,477	26.658	8.658	1.219
`hi_opt_i`	0.449	0.468	9.699	3.114	103,108	21.601	6.936	1.042
`canfield_expert`	0.452	0.443	8.954	2.992	111,680	19.823	6.625	0.982
`hi_opt_ii`	0.505	0.388	7.639	2.764	130,901	15.124	5.472	0.767

This table is not trying to settle the subject forever. It is the current answer under the present study design.

Every Computed Output Column and How It Is Filled

This section is here so a reader can rebuild the sheet without guessing.

Column family	Formula or source
`C1_raw` through `C11_raw`	From the component formulas above
`C1_normalized` through `C11_normalized`	`raw / component_max_in_study`
`instantaneous_subscore`	`C1n + C2n + C3n + C4n + C5n + C7n`
`sustained_subscore`	`C6n + C7n + C8n + C9n + C10n + C11n`
`raw_complexity_total`	Sum of raw `C1` through `C11`
`normalized_complexity_score`	Mean of the twelve normalized slots, with `C7` counted twice
`bjw_ev_per_hand_units`	Copied from `summary.evPerHandUnits`
`bjw_ev_per_initial_bet_pct`	Copied from `summary.evPerInitialBetPct`
`bjw_sd_per_hand_units`	Copied from `summary.sdPerHandUnits`
`bjw_avg_bet_units`	Copied from `betting.avgBetUnits`

Column family	Formula or source
`bjw_sim_score`	Copied from `summary.score` when present, otherwise from `summary.risk_primary.score`
`bjw_di`	Copied from `summary.risk_primary.di`
`bjw_n0`	Copied from `summary.risk_primary.n0`
`bjw_score_per_complexity`	`bjw_sim_score / normalized_complexity_score`
`bjw_di_per_complexity`	`bjw_di / normalized_complexity_score`
`bjw_ev_pct_per_complexity`	`bjw_ev_per_initial_bet_pct / normalized_complexity_score`
`bjw_inverse_n0_per_complexity`	`(1,000,000 / bjw_n0) / normalized_complexity_score`
`qfit_bc`, `qfit_pe`, `qfit_ic`	Published reference values from QFIT
`computed_bc`, `computed_pe`, `computed_ic`	Computed from the static tag vector using the formulas in the BC/PE/IC section
`delta_bc_vs_qfit`, `delta_pe_vs_qfit`, `delta_ic_vs_qfit`	`computed - qfit`
`flags`	Set for every component with a non-zero raw value
`status`, `issues`, `warnings`	Validation fields from the scorer

The Current Study Basis

The current 19-system sheet is not a "full system in all its practical tricks" study. It is a base-count/base-spread study.

Specifically:

- the study rows declare `deviation_package_basis = none`
- the attached `results.json` files show `deviation_precompiler.enabled = false`

So these numbers do not reflect full indexed-play ceilings. They reflect count-and-bet performance under the reference game and the authored spreads used in the study.

That is not a small caveat. It is one of the main ones.

There is also a second caveat sitting above it:

- the current figures are Track B-defensible internal BJW outputs

- they are not a claim of Track A commercial-tool parity

If a system's practical selling point is "it gets much better when you add the usual playing indices" or "it shines when used with a side-count-adjusted package," this study is not yet capturing that ceiling.

Soft Spots and Caveats

This is the section expert readers should read before they get too excited or too annoyed.

1. The normalized score is study-relative

Add harder systems and the denominators move. When the denominators move, every normalized score moves.

That is why `raw_complexity_total` is useful as a stable companion number.

2. `C7` is intentionally double-counted

That is a modeling choice, not a natural constant.

3. Side-count burden is declared, not inferred

That is deliberate. The same base count can exist in materially different operational variants.

4. The current power ratios use BJW's current risk contract

For counted runs, `SCORE`, `DI`, and `N0` are copied from `risk_primary` on a per-round observed basis. If another simulator uses a different basis, the numbers will not be apples-to-apples until you align the basis.

5. `N0` is a convention-sensitive field

Some documents and tools define `N0` as a one-sigma crossing point:

```
N0 = (SD / EV)^2
```

Others define it for 95 percent confidence:

```
N0_95 = (1.96 * SD / EV)^2
```

The current BJW `risk_primary.n0` uses the one-sigma form. Anyone reproducing this with CVData or another tool needs to keep that straight.

6. `computed_pe` is not QFIT PE

It is a stated proxy. It is useful as a transparent internal measure. It is not a published PE reproduction.

7. The current BC and IC alignment is encouraging but limited

The close match to QFIT tells us the tag files and the correlation math are probably right. It does not validate BJW simulation output by itself because those computations come from static tag vectors, not

from simulation traces.

8. Absolute counted values still carry the standard Track A disclosure

This report does compare absolute BJW outputs across systems, because that is necessary to build the power-per-complexity ratios. But those absolute BJW counted outputs still carry the repo's standard disclosure:

- Track B says the math and rule implementation are defensible
- Track A says BJW has not achieved parity with CVData on the canonical external-reference suite

So if this report says one system produces a higher BJW `SCORE per complexity` than another, that is a strong relative statement inside BJW. It is not the same thing as saying the underlying absolute BJW `SCORE` values are externally settled against CVData.

How To Reproduce This Yourself

If someone wants to reproduce the same logic in BJW, CVData, or any other serious simulator, the math path is straightforward.

Step 1: Freeze the comparison basis

Pick one game and do not move it.

That means:

- decks
- rules
- penetration
- play basis
- deviation basis
- spread basis
- minimum rounds

If those are not fixed, the rest of the exercise is just noise in spreadsheet clothing.

Step 2: Define the exact operational variant for each system

For each row, declare:

- tag set
- true-count or running-count operation

- divisor precision convention
- IRC
- side count included or excluded, and what kind
- key count yes or no
- pivot yes or no
- deviation basis

Do not score a count name in the abstract. Score a specific operating variant.

Step 3: Compute the analytical components

From the system definition, compute:

- `C1`
- `C2`
- `C3`
- `C4`
- `C5`
- `C7`
- `C8`
- `C10`
- `C11`

Step 4: Obtain the histogram-derived components

From the simulator, export the running-count distribution at the betting decision point for the fixed reference game.

Then compute:

- `C6 = p99 - p1`
- `C9 = P(RC < 0)`

If your tool cannot export an RC histogram or equivalent per-count frequency table, you cannot reproduce `C6` and `C9` on this exact basis.

Step 5: Normalize across the full study set

Compute the denominator for each component from the maximum raw value observed across the systems included in the study. Then normalize every system against those maxima.

Step 6: Compute the final complexity score

Use the twelve-slot mean with `C7` counted twice.

Step 7: Attach power metrics

From the simulator outputs, extract or compute:

- EV
- average bet
- SD
- SCORE
- DI
- N0

If you use CVData, this is where basis discipline matters. Do not mix:

- one tool's per-hand EV
- another tool's per-round DI
- and a third document's 95-percent `N0`

Pick a contract and stick to it.

Step 8: Compute the "miles per gallon" ratios

Divide the chosen power metric by the normalized complexity score.

The four most useful ratios in the current study are:

- `SCORE / complexity`
- `DI / complexity`
- `EV% / complexity`
- `1/N0 / complexity`

Step 9: Handle BC, PE, and IC honestly

For `BC` and `IC`, there is no reason to be mysterious. Use the tag vector and the explicit target vectors and compute the correlation.

For `PE`, do not bluff.

If you have an exact published target basis you trust, use it and say what it is.

If you only have a proxy, use the proxy and label it as a proxy.

If you want the QFIT table value, the safest thing is to use the QFIT table value and say that is what you are doing.

What This Means

The real value here is not that it gives blackjack one more table.

The value is that it separates three things that are too often blurred together:

- structural difficulty
- practical operating burden
- actual return on that burden

That matters because players do not play abstract counts. They play operating variants, in actual games, with actual arithmetic limits and actual fatigue. A system that looks elegant in a tag table can be clumsy in the shoe. A system that looks complicated on paper can turn out to be a very good bargain once you ask how much `SCORE` it is buying you per unit of complexity. And a system that gets praised for raw power can turn out to be a lousy trade when you finally divide by the trouble.

If this framework is useful, that is why. It does not try to settle every argument in blackjack theory. It tries to force the argument into explicit terms. What game? What variant? What burden? What output? What formula? What caveat? Once those are on the table, people can disagree honestly. And if someone wants to run the same study in CVData, or in their own simulator, they can do it the hard but fair way: freeze the game, define the variant, export the histogram, compute the raw terms, normalize the set, attach the power metrics, and divide power by burden without pretending any of it fell out of the sky.

References

- Framework PDF: Counting System Complexity and Mental Load Scoring Framework.pdf
- QFIT counting summary: https://www.qfit.com/card-counting.htm
- Wizard of Odds effect of removal table: https://wizardofodds.com/games/blackjack/effect-of-removal/